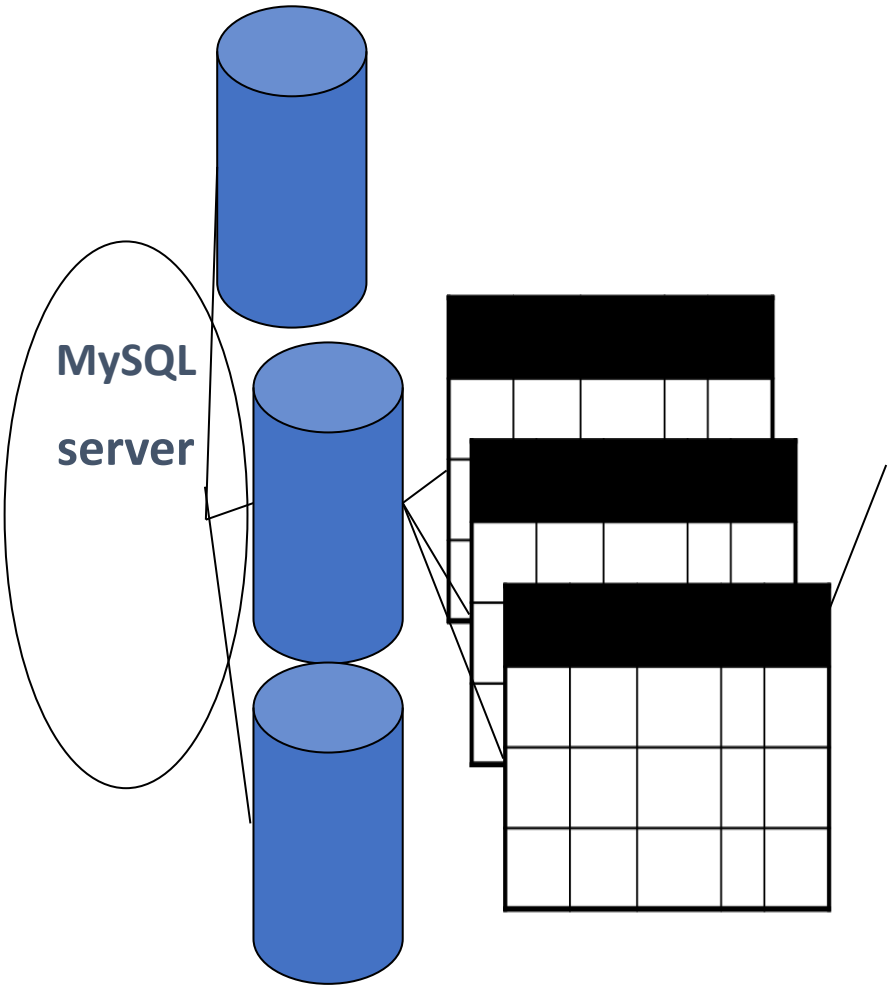
DATABASE ADMINISTRATION



dcp	dt	RH
1	2003-01-01 12:00:00	50.6
2	2003-01-01 12:00:00	45.6
3	2003-01-01 12:00:00	30.9

INTRODUCTION TO MYSQL

MySQL is an SQL (Structured Query Language) based relational database management system (DBMS)

In a relational database:

- A Data element stored in a column with attributes
- A Row is collection of columns
- A Table is a collection rows
- A Database is a collection tables

MySQL is compatible with standard SQL

MySQL is frequently used by PHP and Perl

MySQL server can interact with many databases

MySQL Can run under many operating systems

(FreeBSD, Linux, OS/2 Warp, UNIX, Windows 2000/XP…)

MySQL has both a duel licensing GPL and Commercial

GPL (GNU Public License) free to use if following GPL

Commercial license costs money

MySQL is a very popular, open source database, Handles very large databases, with very fast performance.

Why are we using MySQL?

- Free (much cheaper than Oracle!)
- Each student can install MySQL locally.
- Easy to use Shell for creating tables, querying tables, etc.
- Easy to use with Java JDBC…

Entering & Editing commands

- Prompt mysql>
- issue a command
- Mysql sends it to the server for execution
- displays the results
- prints another mysql>
- a command could span multiple lines

A command normally consists of SQL statement followed by a semicolon.

Command prompt

prompt	meaning
mysql>	Ready for new command.
->	Waiting for next line of multiple-line command.
`'>`	Waiting for next line, waiting for completion of a string that began with a single quote (“'”).
`">`	Waiting for next line, waiting for completion of a string that began with a double quote (“"”).
`` `> ``	Waiting for next line, waiting for completion of an identifier that began with a backtick (“`”).
/*>	Waiting for next line, waiting for completion of a comment that began with /*.

MySQL commands

Help: \h
Quit/exit: \q
Cancel the command: \c …etc

Info about databases and tables

Listing the databases on the MySQL server host

show databases;

Access/change database

Use database_name;

Showing the current selected database

select database();

Showing tables in the current database

show tables;

Showing the structure of a table

describe table_name;

Basic MySQL Operations

Create table
Insert records
Load data
Retrieve records
Update records
Delete records

Modify table
Join table
Drop table
Optimize table
Count, Like, Order by, Group by
More advanced ones (sub-queries, stored procedures, triggers, views …)

Basic Queries

Once logged in, you can try some simple queries. For example:

SELECT VERSION(), CURRENT_DATE;

Note that most MySQL commands end with a semicolon (;)

MySQL returns the total number of rows found, and the total time to execute the query. Keywords may be entered in any lettercase, so the following queries are equivalent:

SELECT VERSION(), CURRENT_DATE;
select version(), current_date;
SeLeCtvErSiOn(), current_DATE;

Mysql architecture

The Mysql database system operates using the client/server architecture. The **server** is the central program that manages database content, and **client programs** connect to the server to retrieve or modify data.

Mysql Architecture overview

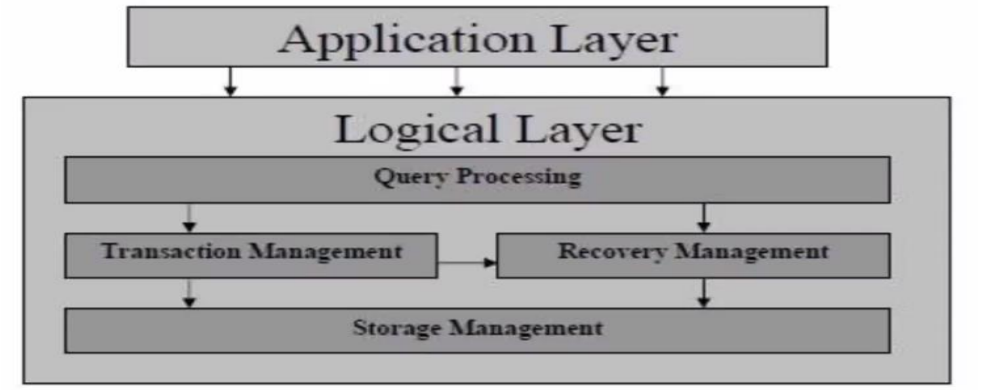
Application Layer:

- Users and clients interact with the Mysql RDBMS
- Three components: **Administrators, clients and query users**
- Query users interact with Mysql RDBMS using **"mysql"**
- Administrators use various administrative interface and utilities like **mysqladmin**
- Clients communicate with the Mysql RDBMS through various interfaces and utilities like the **Mysql APIs**
- **"mysql"** is actually a query interface. It is a monitor that allows users to issue SQL statements and view the results.

Logical Layer:

- The logical layer of Mysql architecture is divided into various subsystems: **Query processor, Transaction Management, Recovery Management and Storage Management.**

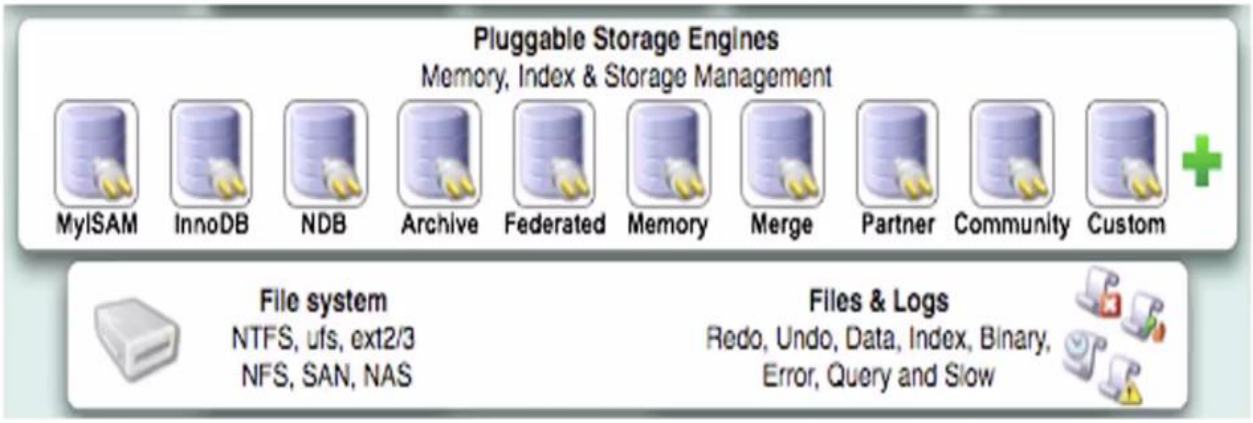
- These sub systems work together to process the requests issued to the Mysql database server.



Phisical Layer:

Mainly deals with storage of a variety of information, which is kept in secondary storage and accessed via the storage manager. The types of data kept in the system are:

- Data file, which stores the user data in the database
- Data dictionary, which stores metadata about the structure of the database
- Indices, which provides fast access to data items that hold particular values
- Statistical data, which store statistical information about the data in the database, it is used by the query processor to select efficient ways to execute a query
- Log information, used to keep track of executed queries such that the recovery manager can use the information to successfully recover the database in the case of a system crash.



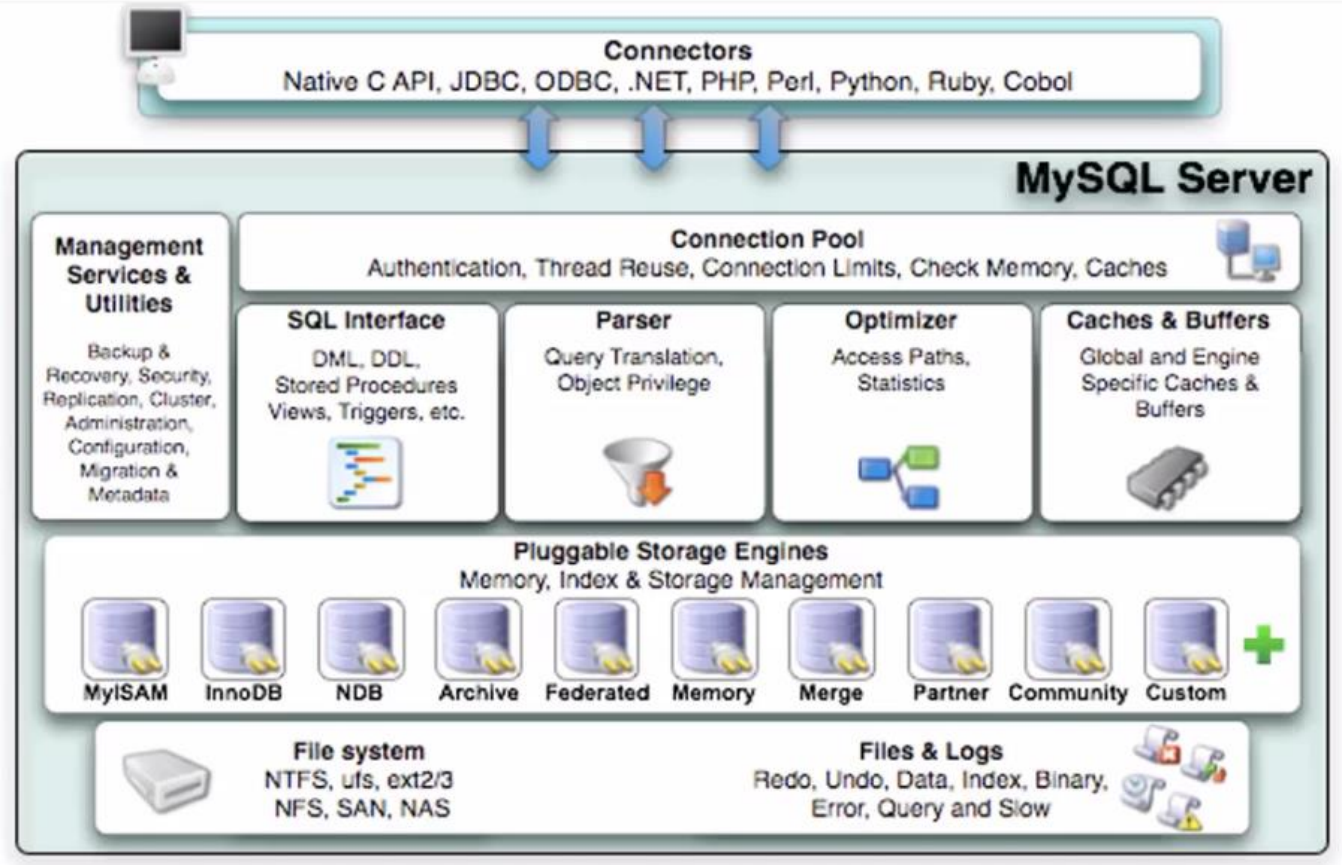
Communication Protocols

- A Mysql client program can connect to a server running on the same machine or another machine

- Mysql supports connections between clients and the server using several network protocols
- Some protocols are applicable for connecting to either local or remote servers. Others can be used only for local servers.

Protocol	Types of Connections	Supported Operating Systems
TCP/IP	Local, remote	All
Unix socket file	Local only	Unix only
Named pipe	Local only	Windows only
Shared memory	Local only	Windows only

Connectors and Mysql Server



Basic Database Server Concepts

- Database runs as a server
  - Attaches to either a default port or an administrator specified port
- Clients connect to database
  - For secure systems
    * authenticated connections
    *usernames and passwords

- Clients make queries on the database
  - Retrieve content
  - Insert content
- **SQL (Structured Query Language)** is the language used to insert and retrieve content.

Database Management System

Manages the storage and retrieval of data to and from the database and hides the complexity of what is actually going on from the user.

MySQL is a relational database management system

phpMyAdmin

- MySQL can be controlled through a simple command-line interface; however, we can use phpMyAdmin as an interface to MySQL.
- **phpMyAdmin** is a very powerful tool; it provides a large number of facilities for customising a database management system.

Data Administration/Administrator (DA)

- ✧ DA (sometimes called *data architect* or even *business analyst)* is a type of professional that resides in the IS function or in a unit interfacing with the IS function.
- ✧ Focus on informing in function of business, users (reports, output forms, queries) rather than IT
  - ✦ Data definition and integration (e.g., Customer entity in CRM systems cutting across Sales, Marketing, R+D…).
  - ✦ Decision support.
  - ✦ Ideas for system design, involvement in system development.
  - ✦ Data governance and security.

Database Administrator (DBA)

- ✧ DBA is focused on technology.

  1. DBA actively participates in DB system life cycle (plan, develop, install, manage, upgrade…).
  2. DBA manages DB system:
     - 2.1 Users: Creating user accounts, assigning use privileges
     - 2.2 System performance: Monitoring and tuning

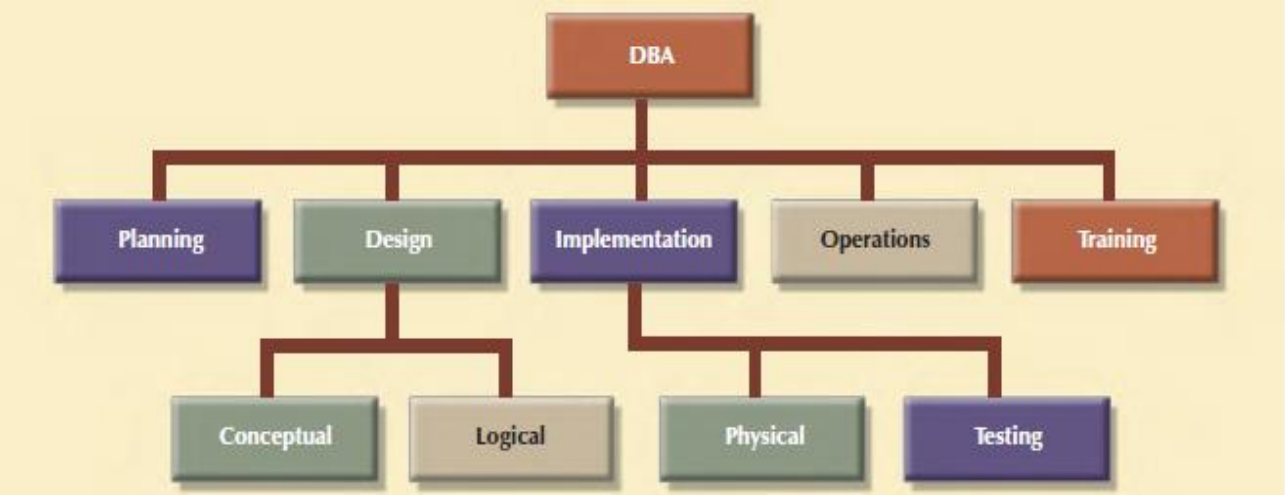
2.3 Backup & recovery: Supervising backups & system restoration after crashes

2.4 Security: Monitoring

Definitions

- ***Data Administration***: A high-level function that is responsible for the overall management of data resources in an organization, including maintaining corporate-wide definitions and standards.
- ***Database Administration***: A technical function that is responsible for physical database design and for dealing with technical issues such as security enforcement, database performance, and backup and recovery.

A DBA functional organization



DBA's Task 1 - System Planning & Design

- Estimation & Design (logical, physical)
  - Data storage requirements, forms & reports needed (costs of development), hardware needs, matching organizational needs with DBMS products
  - Time, labor & cost to develop
  - Data modeling – coordinates with Data Analyst in the domain of logical design (e.g., class diagrams, user interface). Also DA and DBA cooperate on schemas.
  - In charge of technicalities of physical design (types of files, access structures, DBMS product, hardware)

DBA: System Development & Deployment

- Defining technology standards:
  - Programming standards.
    - Layout and techniques.

        - Variable & object definition.
        - User interface.
    - System testing techniques.
- Loading databases.
- Backup and recovery plans.
- User and operator training.

DBA: System Upgrade

- Determines need for change
    - Size and speed of the DB system
    - Usage patterns
    - System output:
        - Additional reports & queries (coop. with DA and business analysts)
    - Forecasting needs

DBA's Task 2.1 - Users' Access

- Control via:
- 1. Operating system
    - Access to directories
    - Access to files
    - Assigned to individuals or groups.
- 2. DBMS functions

(Read, write, modify… data; Administer system)

SQL Security Commands

- GRANT *privileges*
- REVOKE *privileges*
- Privileges include
    - SELECT
    - DELETE
    - INSERT
    - UPDATE
- Objects include
    - Table
    - Table columns
    - Query
- Users include
    - Name/Group

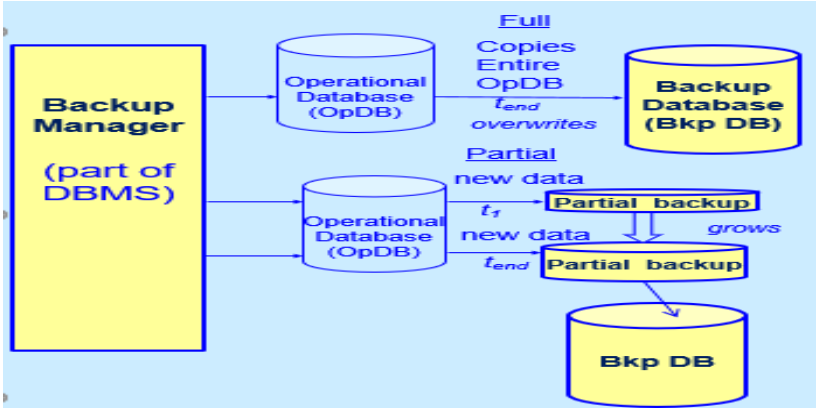
- PUBLIC

DBA: User Identification

- User identification
- Accounts
  - Individual
  - Groups
- Passwords
- Alternative identification
  - Finger & hand print readers
  - Voice…
  - Disposable passwords

DBA's Task 2.3 - Database Backup

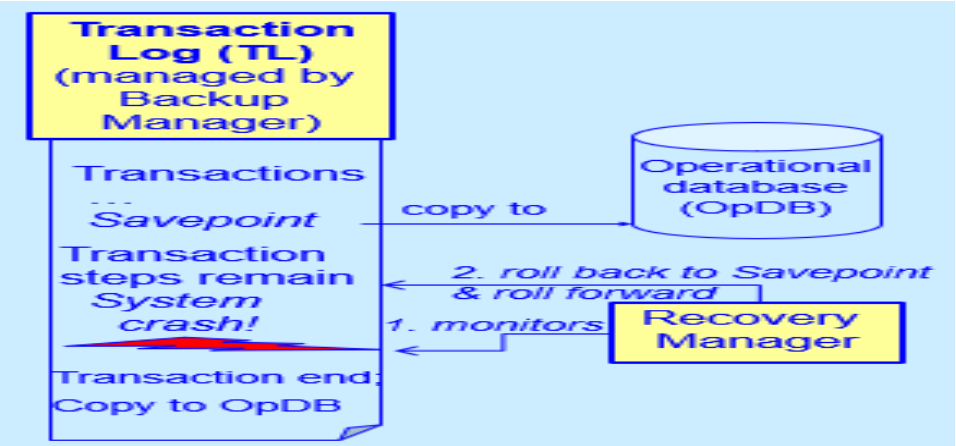
- Backups are crucial
- Offsite storage needed
- Types of backup
  - Full – in longer intervals (e.g., once a week);
    a copy of all tables made
  - Partial (Differential) – in shorter intervals (e.g., day);
    just new data are backed up; more frequent backups reduced loss risk but cost more



DBA: 2.3 Database Recovery

- Recovery needed if there are problems with software, hardware, incorrect user input, viruses, natural causes…
- Recovery (the D-aspect of ACID principle) = durable database in spite of crashes (ex.: transferring and from savings to checking account)
- Key facilities:
  - Recovery Manager (part of DBMS)

- *Transactions log (TL) file*
- ROLLBACK procedure (delete incomplete transaction or part after *Savepoint*.



DBA's Task 2.4 - Database Security

- Security Aspects:

1. Privacy (ownership, access rights & violations)
2. Physical security
   - Protecting hardware
   - Protecting software and data.
3. Logical security
   - Unauthorized actions

Security Threats

- Employees
- Programmers
- Visitors
- Consultants
- Business partnerships
  1. Strategic sharing
  2. EDI (Electronic Data Interchange & other inter-org. networks)
- Hackers (Internet)

STARTING MYSQL AS A WINDOWS SERVICE

On Windows, the recommended way to run MySQL is to install it as a Windows service, whereby MySQL starts and stops automatically when Windows starts and stops. A MySQL server installed as a service can also be controlled from the command line using NET commands, or with the graphical

Services utility. Generally, to install MySQL as a Windows service you should be logged in using an account that has administrator rights

C:\> "C:\Program Files\MySQL\MySQL Server 5.5\bin\mysqladmin" -u root shutdown

Note:

If the MySQL root user account has a password, you need to invoke mysqladmin with the -p option and supply the password when prompted. This command invokes the MySQL administrative utility mysqladmin to connect to the server and tell it to shut down. The command connects as the MySQL root user, which is the default administrative account in the MySQL grant system. Note that users in the MySQL grant system are wholly independent from any login users under Windows.

MySQL client programs that connect to the MySQL server:

MySQL Client Programs

• **mysql:** The command-line tool for interactively entering SQL statements or executing them from a file in batch mode. **"mysql is The MySQL Command-Line Tool".**

• **mysqladmin:** A client that performs administrative operations, such as creating or dropping databases, reloading the grant tables, flushing tables to disk, and reopening log files. mysqladmin can also be used to retrieve version, process, and status information from the server. **"mysqladmin is a Client for Administering a MySQL Server".**

• **mysqlcheck:** A table-maintenance client that checks, repairs, analyzes, and optimizes tables. **"mysqlcheck is a Table Maintenance Program".**

• **mysqldump:** A client that dumps a MySQL database into a file as SQL, text, or XML. **"mysqldump is a Database Backup Program".**

• **mysqlimport:** A client that imports text files into their respective tables using LOAD DATA INFILE. **"mysqlimport is a Data Import Program".**

• **mysqlshow:** A client that displays information about databases, tables, columns, and indexes. **"mysqlshow is a Display Database, Table, and Column Information".**

• **mysqlslap:** A client that is designed to emulate client load for a MySQL server and report the timing of each stage. It works as if multiple clients are accessing the server. **"mysqlslap is a Load Emulation Client".**

You can use mysqladmin to check the server's configuration and current status, to create and drop databases, and more. Invoke mysqladmin like this:

C:\wamp\bin\mysql\mysql5.5.8\bin> mysqladmin [options] command [command-arg] [command [command-arg]] ...

mysqladmin supports the following commands. Some of the commands take an argument following the command name.

- **create db_name:** Create a new database named db_name.
- **debug:** Tell the server to write debug information to the error log.

This includes information about the Event Scheduler."Event Scheduler Status".

- **drop db_name:** Delete the database named db_name and all its tables.
- **extended-status:** Display the server status variables and their values.
- **flush-hosts:** Flush all information in the host cache.
- **flush-logs:** Flush all logs.
- **flush-privileges:** Reload the grant tables (same as reload).
- **flush-status:** Clear status variables.
- **flush-tables:** Flush all tables.
- **flush-threads:** Flush the thread cache.

Connecting to the MySQL Server

For a client program to be able to connect to the MySQL server, it must use the proper connection parameters, such as the name of the host where the server is running and the user name and password of your MySQL account. Each connection parameter has a default value, but you can override them as necessary using program options specified either on the command line or in an option file. The examples here use the mysql client program, but the principles apply to other clients such as mysqldump, mysqladmin, or mysqlshow.

CONFIGURATION OF MYSQL SERVER

Running and Shutting down MySQL Server

First check if your MySQL server is running or not. Now, if you want to shut down an already running MySQL server, then you can do it by using the following command:

```
C:\wamp\bin\mysql\mysql5.5.8\bin> mysqladmin -u root -p shutdown
Enter password: ******
```

SETTING UP A MYSQL USER ACCOUNT

Go to the bin folder of the server installed (WAMP/XAMP …) in drive C, copy the path "C:\wamp\bin\mysql\mysql5.5.8\bin" from the address bar, open your windows command line interface and change the current directory,

From the command prompt: **Cd C:\wamp\bin\mysql\mysql5.5.8\bin**
Connect as root :
C:\wamp\bin\mysql\mysql5.5.8\bin> mysql -u root - p -h localhost
Enter password: ****** (if no password, press enter key)
mysql> use mysql;
Database changed
mysql> INSERT INTO user
(host, user, password, select_priv, insert_priv, update_priv)
VALUES ('localhost', 'guest', 'mypass'), 'Y', 'Y', 'Y');
mysql> FLUSH PRIVILEGES;
The **CREATE USER** command can also enable us to create an account.
To display the user account if the user_name is known:
mysql> SELECT host, user, password FROM user WHERE user = 'guest';
To display all the user account of the localhost:
mysql> SELECT host, user, password FROM user WHERE host = 'localhost';

ADMINISTRATIVE MYSQL COMMAND

Here is the list of the important MySQL commands, which you will use time to time to work with MySQL database:

- **USE Database_name:** This will be used to select a database in the MySQL work area.
- **SHOW DATABASES:** Lists out the databases that are accessible by the MySQL DBMS.
- **SHOW TABLES:** Shows the tables in the database once a database has been selected with the use command.
- **SHOW COLUMNS FROM table_name**: Shows the attributes, types of attributes, key information, whether NULL is permitted, defaults, and other information for a table.
- **SHOW INDEX FROM table_name**: Presents the details of all indexes on the table, including the PRIMARY KEY.
- **SHOW TABLE STATUS LIKE tablename\G**: Reports details of the MySQL DBMS performance and statistics. **Etc...**

MANAGEMENT OF USERS AND SECURITY

CREATE USER Syntax

mysql> CREATE USER 'user_name'@'localhost' IDENTIFIED BY 'password';

The CREATE USER statement creates new MySQL accounts. To use it, you must have the global CREATE USER privilege or the INSERT privilege for the mysql database. For each account, CREATE USER creates a new row in the **mysql.user** table and assigns the account no privileges. An error occurs if the account already exists. If you specify only the user name part of the account name, a host name part of '%' is used. The user specification may indicate how the user should authenticate when connecting to the server:

• To enable the user to connect with no password (which is insecure), include **no** IDENTIFIED BY clause:

```
mysql> CREATE USER 'user_name'@'localhost';
```

RENAME USER Syntax

```
mysql> RENAME USER 'user_name1'@'localhost' TO 'user_name2'@'127.0.0.1';
```

The RENAME USER statement renames existing MySQL accounts. To use it, you must have the global CREATE USER privilege or the UPDATE privilege for the mysql database. An error occurs if any old account does not exist or any new account exists. RENAME USER causes the privileges held by the old user to be those held by the new user. However, RENAME USER does not automatically drop or invalidate databases or objects within them that the old user created.

DROP USER Syntax

```
mysql> DROP USER 'user_name'@'localhost';
```

The DROP USER statement removes one or more MySQL accounts and their privileges. It removes privilege rows for the account from all grant tables. To use this statement, you must have the global CREATE USER privilege or the DELETE privilege for the mysql database.

Note:

DROP USER does not automatically close any open user sessions. Rather, in the event that a user with an open session is dropped, the statement does not take effect until that user's session is closed. Once the session is closed, the user is dropped, and that user's next attempt to log in will fail. This is by design. DROP USER does not automatically drop or invalidate databases or objects within them that the old user created.

GRANT Syntax

The GRANT statement grants privileges to MySQL user accounts. GRANT also serves to specify other account characteristics such as use of secure connections and limits on access to server resources. To use GRANT, you

must have the GRANT OPTION privilege, and you must have the privileges that you are granting.

Normally, a database administrator first uses CREATE USER to create an account, then GRANT to define its privileges and characteristics. For example:

```
mysql> CREATE USER 'user_name'@'localhost' IDENTIFIED BY 'mypass';
mysql> GRANT ALL ON db1.* TO ' user_name '@'localhost';
mysql> GRANT SELECT ON db2.table1 TO 'user_name'@'localhost';
mysql> GRANT USAGE ON *.* TO 'user_name'@'localhost' WITH MAX_QUERIES_PER_HOUR 90;
```

PRIVILEGES SUPPORTED BY MYSQL

The following table summarizes the permissible priv_type privilege types that can be specified for the GRANT and REVOKE statements.

Field	Privilege Name	Users With This Privilege Can
Select_priv	SELECT	Execute a SELECT query to retrieve rows from a table
Insert_priv	INSERT	Execute an INSERT query
Update_priv	UPDATE	Execute an UPDATE query
Delete_priv	DELETE	Execute a DELETE query
Create_priv	CREATE	CREATE databases and tables
Drop_priv	DROP	DROP databases and tables
Reload_priv	RELOAD	Reload/refresh the MySQL server
Shutdown_priv	SHUTDOWN	Shut down a running MySQL server
Process_priv	PROCESS	Track activity on a MySQL server
File_priv	FILE	Read and write files on the server
Grant_priv	GRANT	GRANT other users the same privileges the user possesses
Index_priv	INDEX	Create, edit, and delete table indexes
Alter_priv	ALTER	ALTER tables

References_priv	REFERENCES	Create, edit, and delete foreign key references
Show_db_priv	SHOW DATABASES	View available databases on the server
Super_priv	SUPER	Execute administrative commands
Create_tmp_table_priv	CREATE TEMPORARY TABLES	Create temporary tables
Lock_tables_priv	LOCK TABLES	Create and delete table locks
Execute_priv	EXECUTE	Execute stored procedures
Repl_slave_priv	REPLICATION SLAVE	Read master binary logs in a replication context
Repl_client_priv	REPLICATION CLIENT	Request information on masters and slaves in a replication context

In GRANT statements, the ALL [PRIVILEGES] or PROXY privilege must be named by itself and cannot be specified along with other privileges. ALL [PRIVILEGES] stands for all privileges available for the level at which privileges are to be granted except for the GRANT OPTION and PROXY privileges.

USAGE can be specified to create a user that has no privileges, or to specify the REQUIRE or WITH clauses for an account without changing its existing privileges. MySQL account information is stored in the tables of the mysql database.

Privileges can be granted at several levels, depending on the syntax used for the **ON** clause. For REVOKE, the same ON syntax specifies which privileges to take away. The examples shown here include no IDENTIFIED BY 'password' clause for brevity, but you should include one if the account does not already exist, to avoid creating an insecure account that has no password.

LEVEL OF PRIVILEGES

Global Privileges

Global privileges are administrative or apply to all databases on a given server, the user root has all privileges. To assign global privileges, use **ON** *.* syntax:

```
mysql> GRANT ALL ON *.* TO 'user_name'@'localhost';
mysql> GRANT SELECT, INSERT ON *.* TO 'user_name'@'localhost';
```

The **CREATE TABLESPACE, CREATE USER, FILE, PROCESS, RELOAD, REPLICATION CLIENT, REPLICATION SLAVE, SHOW DATABASES, SHUTDOWN, and SUPER privileges** are administrative and can only be granted globally.

Other privileges can be granted globally or at more specific levels.
MySQL stores global privileges in the **mysql.user** table.

```
mysql> SELECT Host, User, Password FROM mysql.user;
```

Database Privileges

Database privileges apply to all objects in a given database. To assign database-level privileges, use ON db_name.* syntax:

```
mysql> GRANT ALL ON database_name.* TO 'user_name'@'localhost';
mysql> GRANT SELECT, INSERT ON database_name.* TO 'user_name'@'localhost';
mysql> GRANT USAGE ON database_name.* TO 'user_name'@'localhost';
```

If you use ON * syntax (rather than ON *.* and you have selected a default database, privileges are assigned at the database level for the default database. An error occurs if there is no default database.
The CREATE, DROP, EVENT, and GRANT OPTION privileges can be specified at the database level. Table or routine privileges also can be specified at the database level, in which case they apply to all tables or routines in the database. MySQL stores database privileges in the **mysql.db** table**.**
To display the databases granted:

```
mysql> SELECT Host, Db, User FROM mysql.db;
```

Table Privileges

Table privileges apply to all columns in a given table. To assign table-level privileges, use ON db_name.tbl_name syntax:

```
mysql> GRANT ALL ON mydb.mytbl TO 'user_name'@'localhost';
mysql> GRANT SELECT, INSERT ON mydb.mytbl TO 'user_name'@'localhost';
```

If you specify tbl_name rather than db_name.tbl_name, the statement applies to tbl_name in the default database. An error occurs if there is no default database.
The permissible column-level priv_type values are **ALTER, CREATE VIEW, CREATE, DELETE, DROP, GRANT OPTION, INDEX, INSERT, SELECT, SHOW VIEW, TRIGGER, and UPDATE.** MySQL stores table privileges in the **mysql.tables_priv** table**.**

Column Privileges

Column privileges apply to single columns in a given table. Each privilege to be granted at the column level must be followed by the column or columns, enclosed within parentheses.

```
mysql> GRANT SELECT (col1), INSERT (col1,col2) ON mydb.mytbl TO 'user_name'@'localhost';
```

The permissible priv_type values for a column (that is, when you use a column_list clause) are **INSERT, SELECT, and UPDATE**. MySQL stores column privileges in the **mysql.columns_priv** table.

The GRANT Privilege

MySQL lets users grant other users the same privileges they have, via the special **WITH GRANT OPTION** clause of the GRANT command. When this clause is added to a GRANT command, users to whom it applies can assign the same privileges they possess to other users. Consider the following example:

```
mysql> GRANT SELECT, DELETE, INSERT, UPDATE, CREATE, DROP, INDEX ON database_name.* TO 'user_name'@'localhost' WITH GRANT OPTION;
```

The `'user_name'@'localhost'` will now have the GRANTprivilege and can assign his rights to other users, as clearly demonstrated in the following snippet:

```
mysql> SELECT Host, Db, User, Grant_priv FROM db WHERE Host = 'localhost';
```

various commands to display privileges are:

note:

```
mysql> GRANT SELECT, INSERT, UPDATE, DELETE, ON database_name.* TO 'user_name'@'localhost' IDENTIFIED BY 'password';
```

We can accomplish the same task with the following two (equivalent) INSERT commands:

```
mysql> INSERT INTO user (Host, User, Password) VALUES ('localhost', 'user_name','password');
mysql>INSERT INTO db (Host, Db, User, Select_priv, Insert_priv, Update_priv, Delete_priv, Create_priv, Drop_priv) VALUES ('localhost', 'database_name', 'user_name', 'Y', 'Y', 'Y', 'Y', 'N', 'N');
mysql>FLUSH PRIVILEGES;
```

REVOKING PRIVILEGES

REVOKE Syntax

The REVOKE statement enables system administrators to revoke privileges from MySQL accounts. MySQL does not automatically revoke any privileges when you drop a database or table.

```
mysql> REVOKE INSERT ON *.* FROM 'user_name'@'localhost';
```

To use the first REVOKE syntax, you must have the GRANT OPTION privilege, and you must have the privileges that you are revoking. To revoke all privileges, use the second syntax, which drops all global, database, table, column, and routine privileges for the named user or users:

```
mysql> REVOKE ALL PRIVILEGES, GRANT OPTION FROM user [, user] ...
```

To use this REVOKE syntax, you must have the global CREATE USER privilege or the UPDATE privilege for the mysql database. REVOKE removes privileges, but does not drop mysql.user table entries. To remove a user account entirely, use DROP USER or DELETE.

```
mysql> DROP USER user [, user] ...
mysql> DROP USER 'user_name'@'localhost';
```

LIMITING RESOURCE USAGE

New versions of MySQL has the capability to limit resource usage on the MySQL server, on a per-user basis. This is accomplished by the addition of three new fields to the user table: **max_questions, max_updates**, and **max_connections,** which can be used to limit the number of queries, table or record updates, and new connections by a particular user per hour, respectively. These three fields map into three optional clauses to the GRANT command.

The first of these is the MAX_QUERIES_PER_HOUR clause, which limits the number of queries that can be run by a user in an hour. Here's an example:

```
mysql>GRANT SELECT ON *.* TO 'user_name'@'localhost' WITH MAX_QUERIES_PER_HOUR 5;
```

The MAX_QUERIES_PER_HOUR clause controls the total number of queries permitted per hour, regardless of whether these are SELECT, INSERT, UPDATE, DELETE, or other queries. If this is too all-encompassing, you can also set a limit on the number of queries that change the data in the database, via the MAX_UPDATES_PER_HOUR clause, as in the following:

```
mysql> GRANT SELECT ON *.* TO 'user_name'@'localhost' WITH MAX_UPDATES_PER_HOUR 5;
```

The number of new connections opened by the named user(s) in an hour can be controlled via the MAX_CONNECTIONS_PER_HOUR clause, as the following shows.

```
mysql> GRANT SELECT ON *.* TO 'user_name'@'localhost' WITH MAX_CONNECTIONS_PER_HOUR 3;
```

You can also use these clauses in combination with each other. The following is a perfectly valid GRANT:

```
mysql> GRANT SELECT, INSERT, UPDATE, DELETE ON *.* TO 'user_name'@'localhost' WITH MAX_QUERIES_PER_HOUR 50 MAX_UPDATES_PER_HOUR 10 MAX_CONNECTIONS_PER_HOUR 4;
```

Note, such usage limits cannot be specified per-database or per-table. They can only be specified in the global context, by using an ON *.* clause in the GRANT command.

VIEWING PRIVILEGES

MySQL enables you to view the privileges assigned to a particular user with the **SHOW GRANTS** command, which accepts a username as argument and displays a list of all the privileges granted to that user. The following examples illustrate this:

```
mysql> SHOW GRANTS FOR 'user_name'@'localhost';
mysql> SHOW GRANTS FOR 'root'@'localhost';
mysql> SELECT Host, Db, User, Grant_priv, Select_priv, Delete_priv, Update_priv, Create_priv FROM db WHERE Host = 'localhost';
```

The mysql tables structure

```
mysql> SHOW CREATE TABLE user;
mysql> SHOW CREATE TABLE db;
mysql> SHOW CREATE TABLE `host`;
mysql> SHOW CREATE TABLE columns_priv;
mysql> SHOW CREATE TABLE tables_priv;
```

Reloading the Grant Tables

Privileges set using GRANT and REVOKE are immediately activated (as demonstrated in one of the examples in the preceding section). Privileges set via regular SQL queries, however, require a server reload to come into effect. A server reload can be accomplished via the FLUSH PRIVILEGES command, as in the following:

```
mysql> FLUSH PRIVILEGES;
```

NOTE You need the RELOAD privilege to run the FLUSH command.

```
C:\wamp\bin\mysql\mysql5.5.8\bin> mysqladmin -u root reload
```

BACKUP AND RECOVERY (EXPORTING/IMPORTING DATA)

It is important to back up your databases so that you can recover your data and be up and running again in case problems occur, such as system crashes, hardware failures, or users deleting data by mistake. Backups are also essential as a safeguard before upgrading a MySQL installation, and they can be used to transfer a MySQL installation to another system or to set up replication slave servers. MySQL offers a variety of backup strategies from which you can choose the methods that best suit the requirements for your installation. This section discusses several backup and recovery topics with which you should be familiar:

- Types of backups: **Logical versus physical, full versus incremental, and so forth**
- Methods for creating backups

• Recovery methods, including point-in-time recovery
• Backup scheduling, compression, and encryption
• Table maintenance, to enable recovery of corrupt tables

Making Backups with mysqldump

C:\wamp\bin\mysql\mysql5.5.8\bin>mysqldump

Usage: mysqldump [OPTIONS] database [tables]
OR mysqldump [OPTIONS] --databases [OPTIONS] DB1 [DB2 DB3...]
OR mysqldump [OPTIONS] --all-databases [OPTIONS]

By default, mysqldump writes information as SQL statements to the standard output. You can save the output in a file:

C:\wamp\bin\mysql\mysql5.5.8\bin> mysqldump [arguments] > path\file_name.sql (example of path: **C:\Users\HP\Desktop\FOLDER**)

To dump all databases, invoke mysqldump with the --all-databases option:

C:\wamp\bin\mysql\mysql5.5.8\bin> mysqldump --all-databases > path\dump.sql

To dump only specific databases, name them on the command line and use the --databases option:

C:\wamp\bin\mysql\mysql5.5.8\bin> mysqldump --databases db1 db2 db3 > path\dump.sql

To dump a single database, name it on the command line:

C:\wamp\bin\mysql\mysql5.5.8\bin> mysqldump --databases test > path\dump.sql **or**

C:\wamp\bin\mysql\mysql5.5.8\bin> mysqldump test > path\dump.sql

The difference between the two preceding commands is that without --databases, the dump output contains no CREATE DATABASE or USE statements.

To dump only specific tables from a database, name them on the command line following the database name:

C:\wamp\bin\mysql\mysql5.5.8\bin> mysqldump test t1 t3 t7 > path\dump.sql

Reloading SQL-Format Backups

To reload a dump file written by mysqldump that consists of SQL statements, use it as input to the mysql client. If the dump file was created by mysqldump with the --all-databases or --databases option, it contains CREATE DATABASE and USE statements and it is not necessary to specify a default database into which to load the data:

C:\wamp\bin\mysql\mysql5.5.8\bin> mysql < path\dump.sql

If the file is a single-database dump not containing CREATE DATABASE and USE statements, create the database first (if necessary):

```
C:\wamp\bin\mysql\mysql5.5.8\bin> mysqladmin create db1
```

Then specify the database name when you load the dump file:

```
C:\wamp\bin\mysql\mysql5.5.8\bin> mysql db1 < path\dump.sql
```

Making a Copy of a Database

```
C:\wamp\bin\mysql\mysql5.5.8\bin> mysqldump db1 > path\dump.sql
C:\wamp\bin\mysql\mysql5.5.8\bin> mysqladmin create db2
C:\wamp\bin\mysql\mysql5.5.8\bin> mysql db2 < path\dump.sql
```

Alternatively, from within mysql, use a source command:

```
mysql> source path\dump.sql
```

TABLE MAINTENANCE STATEMENTS

ANALYZE TABLE Syntax

```
mysql> analyze table table_name;
```

ANALYZE TABLE analyzes and stores the key distribution for a table. During the analysis, the table is locked with a read lock for MyISAM and InnoDB. This statement works with **MyISAM** and **InnoDB** tables.

This statement requires SELECT and INSERT privileges for the table. ANALYZE TABLE is supported for partitioned tables, and you can use ALTER TABLE ... ANALYZE PARTITION to analyse one or more partitions.

CHECK TABLE Syntax

```
mysql> check table table_name;
```

CHECK TABLE checks a table or tables for errors. CHECK TABLE works for MyISAM, InnoDB, ARCHIVE, and CSV tables. For MyISAM tables, the key statistics are updated as well. To check a table, you must have some privilege for it. CHECK TABLE can also check views for problems, such as tables that are referenced in the view definition that no longer exist. CHECK TABLE is supported for partitioned tables, and you can use ALTER TABLE ... CHECK PARTITION to check one or more partitions;

CHECKSUM TABLE Syntax

```
mysql> CHECKSUM TABLE tbl_name [, tbl_name] ... [ QUICK | EXTENDED ]
```

CHECKSUM TABLE reports a table checksum. This statement requires the SELECT privilege for the table. With QUICK, the live table checksum is reported if it is available, or NULL otherwise. This is very fast. A live checksum is enabled by specifying the CHECKSUM=1 table option when you create the table; currently, this is supported only for MyISAM tables.

OPTIMIZE TABLE Syntax

mysql> optimize table table_name;

OPTIMIZE TABLE should be used if you have deleted a large part of a table or if you have made many changes to a table with variable-length rows (tables that have VARCHAR, VARBINARY, BLOB, or TEXT columns). Deleted rows are maintained in a linked list and subsequent INSERT operations reuse old row positions. You can use OPTIMIZE TABLE to reclaim the unused space and to defragment the data file. After extensive changes to a table, this statement may also improve performance of statements that use the table, sometimes significantly. This statement requires SELECT and INSERT privileges for the table.

OPTIMIZE TABLE is supported for partitioned tables, and you can use ALTER TABLE ... OPTIMIZE PARTITION to optimize one or more partitions; OPTIMIZE TABLE works only for MyISAM, InnoDB, and ARCHIVE tables. It does not work for tables created using any other storage engine.

For MyISAM tables, OPTIMIZE TABLE works as follows:

1. If the table has deleted or split rows, repair the table.
2. If the index pages are not sorted, sort them.
3. If the table's statistics are not up to date (and the repair could not be accomplished by sorting the index), update them.

For InnoDB tables, OPTIMIZE TABLE is mapped to ALTER TABLE, which rebuilds the table to update index statistics and free unused space in the clustered index. This is displayed in the output of OPTIMIZE TABLE when you run it on an InnoDB table.

REPAIR TABLE Syntax

mysql> repair table table_name;

REPAIR TABLE repairs a possibly corrupted table. REPAIR TABLE works for MyISAM, ARCHIVE, and CSV tables. This statement requires SELECT and INSERT privileges for the table.

REPAIR TABLE is supported for partitioned tables. However, the USE_FRM option cannot be used with this statement on a partitioned table.

You can use ALTER TABLE ... REPAIR PARTITION to repair one or more partitions; Normally, you should never have to run REPAIR TABLE. However, if disaster strikes, this statement is very likely to get back all your data from a MyISAM table. If your tables become corrupted often, you should try to find the reason for it, to eliminate the need to use REPAIR TABLE.

To show the status of the table:
mysql> show table status like 'table_name'\G;

MYSQL STORAGE ENGINES

Data in MySQL is stored in files (or memory) using a variety of different techniques. Each of these techniques employs different storage mechanisms, indexing facilities, locking levels and ultimately provides a range of different functions and capabilities. By choosing a different technique you can gain additional speed or functionality benefits that will improve the overall functionality of your application. Each of these different techniques and suites of functionality within the MySQL system is referred to as a **storage engine** (also known as a table type).

By default, MySQL comes with a number of different storage engines pre-configured and enabled in the MySQL server. You can select the storage engine to use on a server, database and even table basis, providing you with the maximum amount of flexibility when it comes to choosing how your information is stored, how it is indexed and what combination of performance and functionality you want to use with your data.

This flexibility to choose how your data is stored and indexed is a major reason why MySQL is so popular; other database systems, including most of the commercial options, support only a single type of database storage.

A MySQL storage engine is a low-level engine inside the database server that takes care of storing and retrieving data, and can be accessed through an internal MySQL API or, in some situations be accessed directly by an application. Note that one application can have more than one storage engine in use at any given time.

InnoDB is the mostly widely used storage engine for Web/Web 2.0, eCommerce, Financial Systems, Telecommunications, Health Care and Retail applications built on MySQL. InnoDB provides highly efficient ACID-compliant transactional capabilities and includes unique architectural elements that deliver high performance and scalability. InnoDB is structurally designed to handle transactional applications that require crash recovery, referential integrity, high levels of user concurrency and fast response times.

While **MyISAM** and the other storage engines continue to be readily available, users can now create applications built on InnoDB without altering

default configuration settings. All of the capabilities of InnoDB are now delivered "out of the box" with any MySQL 5.5 deployment.

The ACID properties of InnoDB are configurable, and so users can ensure ACID-compliance for those workloads demanding the highest levels of data integrity, suchas ecommerce, while relaxing ACID properties where throughput is more important. Even with these relaxed properties, benefits such as crash recovery are still maintained by InnoDB, so users enjoy much higher levels of protection than they do with MyISAM.

InnoDB to MyISAM Feature Comparison

Feature	InnoDB	MyISAM
ACID Transactions	Yes	No
Configurable ACID Properties	Yes	No
Crash Safe	Yes	No
Foreign Key Support	Yes	No
Row-Level Locking Granularity	Yes	No (Table)
MVCC	Yes	No
Geospatial Data Type Support	Yes	Yes
Geospatial (R-Tree) Indexing Support	No	Yes
B-tree Indexes	Yes	Yes
Full-text search Indexes	No	Yes
Clustered Index	Yes	No
Data Caches	Yes	No
Index Caches	Yes	Yes
Compressed Data	Yes [b]	Yes [a]
Read & Write to Compressed Tables	Yes	No (Read-Only)
Encrypted Data [c]	Yes	Yes
Replication Support [d]	Yes	Yes
Backup / Point-in-Time Recovery [d]	Yes	Yes
Query Cache Support	Yes	Yes
Update Statistics for Data Dictionary	Yes	Yes
Storage Limits (Table Size)	64TB	256TB

[a] Compressed MyISAM tables are supported only when using the compressed row format.

[b] Compressed InnoDB tables require the InnoDB Barracuda file format.

[c] Implemented in the server (via encryption functions), rather than in the storage engine.

[d] Implemented in the server, rather than in the storage engine.

InnoDB delivers the best blend of performance, reliability and data integrity for transactional workloads, with a host of parameters that are configurable, enabling users to optimize performance and behavior for their specific workload.

MyISAM remains a strategic technology for MySQL and its users, and there are some use cases that are better suited to the characteristics of MyISAM. Full-text search indexes in MyISAM are useful for many simple read-only web applications, though often users deploy MySQL and InnoDB with Sphinx or Lucene for fast text searches as an alternative to MyISAM. It is often the case that performing full text searches outside of the database will deliver higher levels of performance and scalability.

Other use cases that are potentially suitable for MyISAM include:

- Applications demanding very high raw INSERT speeds where concurrency is not a consideration. Performance will always be application dependent, so benchmarking is necessary to determine the best solution for your own environment.
- Caches or temporary tables.
- Blogs / Wikis / RSS feeds.
- Read-only tables.

Determining Available Engines

You can determine a list of engines by using the show engines command within MySQL

```
mysql>  show engines;
mysql> show variables like "have_%";
```

Using an Engine

There are a number of ways you can specify the storage engine to use. The simplest method, if you have a preference for a engine type that fits most of your database needs to set the default engine type within the MySQL configuration file (using the option storage_engine or when starting the database server by supplying the --default-storage-engine or --default-table-type options on the command line). More flexibility is offered by allowing you specify the storage engine to be used MySQL, the most obvious is to specify the engine type when creating the table:

```
mysql> CREATE TABLE mytable (id int, title varchar(20)) ENGINE = MyISAM;
mysql> ALTER TABLE mytable ENGINE = INNODB;
```

However, you should be careful when altering table types in this way as making a modification to a type that does not support the same indexes, field types or sizes may mean that you lose data. If you specify a storage engine that doesn't exist in the current database then a table of type MyISAM (the default) is created instead.

MySQL DATA TYPES

DATE TYPE	SPEC	DATA TYPE	SPEC
CHAR	String (0 - 255)	INT	Integer (-2147483648 to 214748-3647)
VARCHAR	String (0 - 255)	BIGINT	Integer (-9223372036854775808 to 9223372036854775807)
TINYTEXT	String (0 - 255)	FLOAT	Decimal (precise to 23 digits)
TEXT	String (0 - 65535)	DOUBLE	Decimal (24 to 53 digits)
BLOB	String (0 - 65535)	DECIMAL	"DOUBLE" stored as string
MEDIUMTEXT	String (0 - 16777215)	DATE	YYYY-MM-DD
MEDIUMBLOB	String (0 - 16777215)	DATETIME	YYYY-MM-DD HH:MM:SS
LONGTEXT	String (0 - 4294967295)	TIMESTAMP	YYYYMMDDHHMMSS
LONGBLOB	String (0 - 4294967295)	TIME	HH:MM:SS
TINYINT	Integer (-128 to 127)	ENUM	One of preset options
SMALLINT	Integer (-32768 to 32767)	SET	Selection of preset options
MEDIUMINT	Integer (-8388608 to 8388607)	BOOLEAN	TINYINT(1)

MYSQL TRANSACTIONS

Introducing transactions

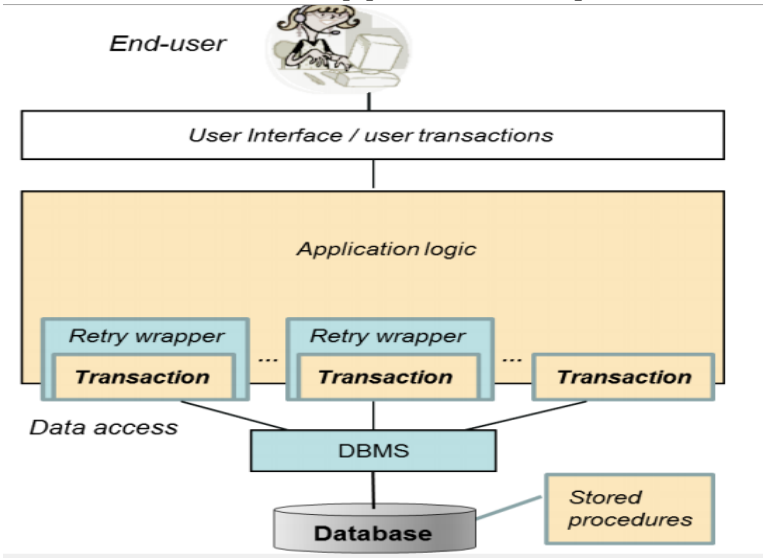
In everyday life, people conduct different kind of business transactions buying products, ordering travels, changing or cancelling orders, buying tickets to concerts, paying rents, electricity bills, insurance invoices, etc. Transactions do not relate only to computers, of course. Any type of human activity comprising a **logical unit of work** meant to either be executed as a whole or to be cancelled in its entirety comprises a **transaction**. Almost all information systems utilize the services of some database management system (DBMS) for storing and retrieving data. Today's DBMS products are technically sophisticated securing data integrity in their databases and providing fast access to data even to multiple concurrent users. They provide reliable services to applications for managing persistency of data, but only if applications use these reliable services properly. This is done by building the **data access** parts of the application logic using **database transactions**. Improper transaction management and control by the application software may, for example, result in

- customer orders, payments, and product shipment orders being lost in the case of a web store
- failures in the registration of seat reservations or double-bookings to be made for train/airplane passengers
- lost emergency call registrations at emergency response centers etc.

Problematic situations like the above occur frequently in real life, but the people in charge often prefer not to reveal the stories to the public.

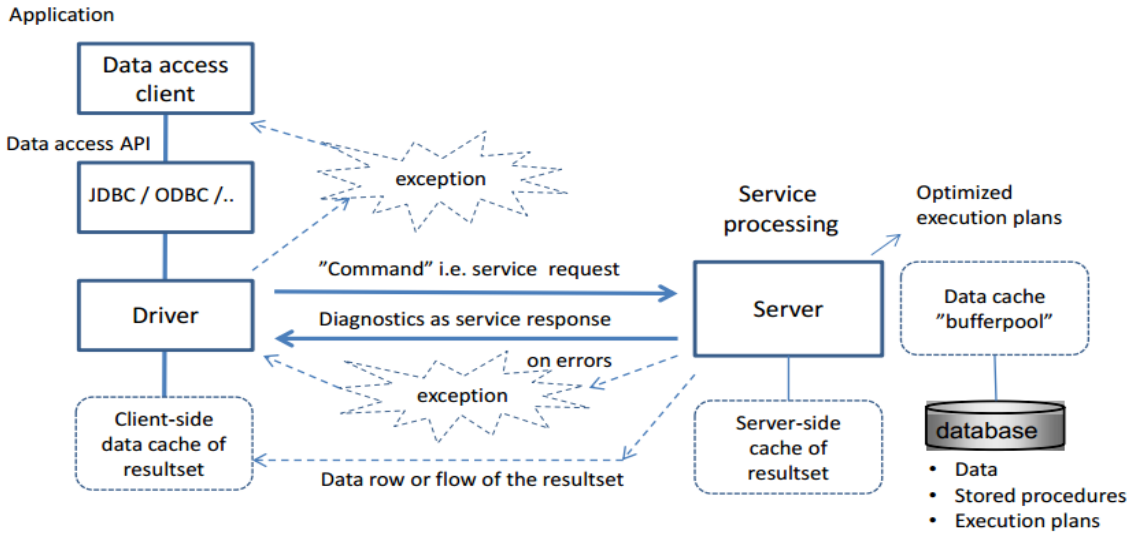
Transactions are recoverable units of data access tasks in terms of database content manipulation. They also comprise units of recovery for the entire database in case of system crashes. They also provide basis for concurrency management in multi-user environment.

Position of SQL transactions in application layers



The above Figure presents a simplified view of the architecture of a typical database application, positioning the database transactions on a different software layer than the user interface layer. From the end user's point of view, for processing a business transaction one or more use cases can be defined for the application and implemented as **user transactions**. A single user transaction may involve multiple SQL transactions, some of which will involve retrieval, and usually the final transaction in the series updating the contents in the database. **Retry wrappers** in the application logic comprise the means for implementing programmatic retry actions in case of concurrency failures of SQL transactions.

To properly understand SQL transactions, we need to agree on some basic concepts concerning the client-server handshaking dialogue. To access a database, the application needs to initiate a **database connection** which sets up the context of an **SQL-session.** For simplicity, the latter is said to comprise the **SQL-client,** and the database server comprises the **SQL-server**. From the database server point of view, the application uses database services in client/server mode by passing **SQL commands** as parameters to functions/methods via a data access API (application programming interface). Regardless of the data access interface used, the "logical level" dialog with the server is based on the SQL language, and reliable data access is materialized with the proper use of SQL transactions.



SQL command processing explained

The Figure above explains the "round trip", the processing cycle of an SQL command, started by the client as a **service request** to the server using a middleware stack and network services, for processing by server, and the returned response to the request. The SQL command may involve one or more **SQL statements**. The SQL statement(s) of the command are parsed, analyzed on the basis of the database metadata, then optimized and finally executed. To improve the performance degradation due to slow disk I/O operations, the server retains all the recently used rows in a RAM-residing buffer pool and all data processing takes place there.

The execution of the entered SQL command in the server is **atomic** in the sense that the whole SQL command has to **succeed**; otherwise the whole

command will be **cancelled** (rolled back). As a response to the SQL command, the server sends diagnostic message(s) reporting of the success or failures of the command. Command execution errors are reflected to the client as a sequence of exceptions.

SQL Transactions

When the application logic needs to execute a sequence of SQL commands in an atomic fashion, then the commands need to be grouped as a logical unit of work (LUW) called SQL transaction which, while processing the data, transforms the database from a consistent state to another consistent state, and thus be considered as **unit of consistency**. Any successful execution of the transaction is ended by a **COMMIT** command, whereas unsuccessful execution need to be ended by a **ROLLBACK** command which automatically recovers from the database all changes made by the transaction. Thus SQL transaction can also be considered as **unit of recovery**. The advantage of the ROLLBACK command is that when the application logic programmed in the transaction cannot be completed, there is no need for conducting a series of reverse operations command-by-command, but the work can be cancelled simply by the ROLLBACK command, the operation of which will always succeed. Uncommitted transactions in case of broken connections, or end of program, or at system crash will be automatically rolled back by the system. Also in case of concurrency conflicts, some DBMS products will automatically rollback a transaction, as explained below.

Some DBMS products, for example, SQL Server, **MySQL/InnoDB**, PostgreSQL operate by default in the **AUTOCOMMIT mode**. This means that the result of every single SQL command will is automatically committed to the database, thus the effects/changes made to the database by the statement in question **cannot be rolled back**. So, in case of errors the application needs do reverse-operations for the logical unit of work, which may be impossible after operations of concurrent SQL-clients. Also in case of broken connections the database might be left in inconsistent state. In order to use real transactional logic, one needs to start every new transaction with an explicit start command, like: BEGIN WORK, BEGIN TRANSACTION, or START TRANSACTION, depending on the DBMS product used.

In **MySQL/InnoDB**, an ongoing SQL session can be set to use either implicit or explicit transactions by executing the statement:
SET AUTOCOMMIT = 0; or **SET AUTOCOMMIT = 1;**
where 0 implies the use of implicit transactions, and 1 means operating in AUTOCOMMIT mode. To display the status of the autocommit variable, we use **SHOW variables like 'autocommit';**

Concurrent Transactions

Concurrency Problems: Possible Risks of Reliability

Without having proper concurrency control services in the database product or lacking knowledge on how to use the services properly, the **content** in the database or **results** to our queries might become corrupted i.e. **unreliable**. In the following, we cover the typical concurrency problems (anomalies):

- **The lost update problem**
- **The dirty read problem**, i.e. reading of uncommitted data of some concurrent transaction
- **The non-repeatable read problem**, i.e. repeated read may not return all same rows
- **The phantom read problem**, i.e. during the transaction some qualifying rows may not be seen by the transaction.

The ACID Principle of Ideal Transaction

The ACID principle, presented by Theo Härder and Andreas Reuter in 1983 at ACM Computing Surveys, defines the ideal of reliable SQL transactions in multi-client environment. The ACID acronym comes from initials of the following four transaction properties:

- **Atomic:** A transaction needs to be an atomic ("All or nothing") series of operations which either succeeds and will be committed or all its operations will be rolled back from the database.
- **Consistent:** The series of operations will transfer the contents of the database from a consistent state to another consistent state. So, at latest, at the commit time, the operations of the transaction have not violated any database constraints (primary keys, unique keys, foreign keys, checks).

- **Isolated:** The original definition by Härder and Reuter, "Events within a transaction must be hidden from other transactions running concurrently" will not be fully satisfied by most DBMS products.
- **Durable:** The committed results in the database will survive on disks in spite of possible system failures.

The ACID principle requires that a transaction which does not fulfil these properties shall not be committed, but either the application or the database server has to rollback such transactions.

Transaction Isolation Levels

The Isolated property in ACID principle is challenging. Depending on the concurrency control mechanisms, it can lead to concurrency conflicts and too long waiting times, slowing down the production use of the database.

The ISO SQL standard does not define how the concurrency control should be implemented, but based on the concurrency anomalies i.e. bad behaving phenomena which we have illustrated above, it defines the isolation levels which need to solve these anomalies as defined in Table below. Some of the isolation levels are less restrictive and some are more restrictive giving better isolation, but perhaps at the cost of performance slowdown. It is worth noticing that isolation levels say nothing about write restrictions. For write operations some lock protection is typically needed, and a successful write is always protected against overwriting by others up to the end of transaction.

ISO SQL transaction isolation levels solving concurrency anomalies

Phenomena: / Isolation Level:	Lost Update	Dirty Read	Non-Repeatable Read	Phantoms
READ UNCOMMITTED	NOT possible	possible !	possible !	possible !
READ COMMITTED	NOT possible	NOT possible	possible !	possible !
REPEATABLE READ	NOT possible	NOT possible	NOT possible	possible !
SERIALIZABLE	NOT possible	NOT possible	NOT possible	NOT possible

Transaction Isolation Levels of ISO SQL (and DB2) explained

ISO SQL Isolation level	DB2 Isol. level	Isolation service explained
Read Uncommitted	UR	allows "dirty" reading of uncommitted data written by concurrent transactions.
Read Committed	CS (CC)	does not allow reading of uncommitted data of concurrent transactions. Oracle and DB2 (9.7 and later) will read the latest committed version of the data (in DB2 this is also called "Currently Committed", CC), while some products will wait until the data gets committed first.
Repeatable Read	RS	allows reading of only committed data, and it is possible to repeat the reading without any UPDATE or DELETE changes made by the concurrent transactions in the set of accessed rows.
Serializable	RR	allows reading of only committed data, and it is possible to repeat the reading without any INSERT, UPDATE, or DELETE changes made by the concurrent transactions in the set of accessed tables.

Concurrency Control Mechanisms

Modern DBMS products are mainly using the following Concurrency Control (CC) mechanisms for isolation

- Multi-Granular Locking scheme (called MGL or just LSCC)
- Multi-Versioning Concurrency Control (called MVCC)
- Optimistic Concurrency Control (OCC).

Implementation of concurrency transaction with different level of isolation

Concurrency issues

Client1:

```
Show databases;
Use company;
Show table;
START TRANSACTION;
SELECT * from employee;
```

Client2

```
Show databases;
Use company;
Show table;
SELECT * from employee;
SELECT * from employee where emp_id=101;!
UPDATE employee SET salary = 175000 WHERE emp_id=101;
SELECT * from employee;
```

Client1

```
UPDATE employee SET salary = (salary +5000) WHERE emp_id=101;
SELECT * from employee;
COMMIT;
SELECT * from employee;
```

```
Show create table table_name;
show variables like '%commit%';
select * from employee
limit 1;
show variables like 'autocommit';
START TRANSACTION;
update employee set salary=150000 where emp_id=100;
select * from employee
 limit 1;
rollback;
select * from employee
 limit 1;

START TRANSACTION;
select * from employee
 limit 1;
update employee set salary=175000 where emp_id=100;
SAVEPOINT SP1;
update employee set salary=180000 where emp_id=100;
select * from employee where emp_id=100;
rollback to SP1;
```

```
select * from employee where emp_id=100;
commit;

show variables like 'tx_isolation';
prompt transaction#1>
use company;
show tables;
select * from employee limit 1;
START TRANSACTION;
update employee set salary=200000 where emp_id=100;
prompt transaction#2>
show variables like 'tx_isolation';
SET tx_isolation = "READ-UNCOMMITTED";
show variables like 'tx_isolation';
use company;
START TRANSACTION;
select * from employee limit 1;

transaction#1>
update employee set salary=250000 where emp_id=100;
transaction#2>
select * from employee limit 1;
transaction#1>
rollback;
transaction#2>
select * from employee limit 1;
transaction#1>
START TRANSACTION;
select * from employee limit 1;
transaction#2>
show variables like 'tx_isolation';
SET tx_isolation = "REPEATABLE-READ";
show variables like 'tx_isolation';
START TRANSACTION;
select * from employee limit 1;
```

transaction#1>
update employee set salary=500000 where emp_id=100;
select * from employee limit 1;
transaction#2>
select * from employee limit 1;
transaction#1>
commit;
transaction#2>
select * from employee limit 1;
commit;
select * from employee limit 1;
transaction#1>
START TRANSACTION;
select * from employee limit 1;
transaction#2>
show variables like 'tx_isolation';
SET tx_isolation = "SERIALIZABLE";
show variables like 'tx_isolation';
START TRANSACTION;
select * from employee limit 1;
transaction#1>
update employee set salary=600000 where emp_id=100;
select * from employee limit 1;
transaction#2>
commit;
transaction#1>
commit;

PARTITIONING

This section discusses MySQL's implementation of user-defined partitioning. You can determine whether your MySQL Server supports partitioning by means of a SHOW VARIABLES statement such as this one:

mysql> SHOW VARIABLES LIKE '%partition%';

You can also check the output of the SHOW PLUGINS statement, as shown here:

mysql> SHOW PLUGINS;

If you do not see the have_partitioning variable with the value YES listed in the output of an appropriate SHOW VARIABLES statement, or if you do not see the partition plugin listed with the value ACTIVE for the Status column in the output of SHOW PLUGINS, then your version of MySQL was not built with partitioning support.

Partitioning by allows you to distribute portions of individual tables across a filesystem according to rules which you can set largely as needed. In effect, different portions of a table are stored as separate tables in different locations. The user-selected rule by which the division of data is accomplished is known as a **partitioning function**, which in MySQL can be the modulus, simple matching against a set of ranges or value lists, an internal hashing function, or a linear hashing function. The function is selected according to the partitioning type specified by the user, and takes as its parameter the value of a user-supplied expression. This expression can be either an integer column value, or a function acting on one or more column values and returning an integer. The value of this expression is passed to the partitioning function, which returns an integer value representing the number of the partition in which that particular record should be stored. This function must be non-constant and non-random. It may not contain any queries, but may use virtually any SQL expression that is valid in MySQL, so long as that expression returns a positive integer less than MAXVALUE (the greatest possible positive integer).

Partitioning Types

RANGE Partitioning

A table that is partitioned by range is partitioned in such a way that each partition contains rows for which the partitioning expression value lies within a given range. Ranges should be contiguous but not overlapping, and are defined using the VALUESLESS THAN operator. For the next few examples, suppose that you are creating a table such as the following to hold personnel records for a chain of 20 video stores, numbered 1 through 20:

```
CREATE TABLE employees (
id INT NOT NULL,
fname VARCHAR(30),
lname VARCHAR(30),
hired DATE NOT NULL DEFAULT '1970-01-01',
separated DATE NOT NULL DEFAULT '9999-12-31',
job_code INT NOT NULL,
store_id INT NOT NULL
);
```

This table can be partitioned by range in a number of ways, depending on your needs. One way would be to use the store_id column. For instance, you might decide to partition the table 4 ways by adding a PARTITIONBY RANGE clause as shown here:

```
ALTER TABLE employees PARTITIONBY RANGE (store_id) (
PARTITION p0 VALUESLESS THAN (6),
PARTITION p1 VALUESLESS THAN (11),
PARTITION p2 VALUESLESS THAN (16),
PARTITION p3 VALUESLESS THAN MAXVALUE
);
```

MAXVALUE represents an integer value that is always greater than the largest possible integer value (in mathematical language, it serves as a least upper bound).

LIST Partitioning

List partitioning is similar to range partitioning in many ways. As in partitioning by RANGE, each partition must be explicitly defined. The chief difference between the two types of partitioning is that, in list partitioning, each partition is defined and selected based on the membership of a column value in one of a set of value lists, rather than in one of a set of contiguous ranges of values. This is done by using PARTITIONBY LIST(expr)where expr is a column value or an expression based on a column value and returning an integer value, and then defining each partition by means of a VALUES IN (value_list), where value_list is a comma-separated list of integers. For the examples that follow,

we assume that the basic definition of the table to be partitioned is provided by the CREATE TABLE statement shown here:

```
CREATE TABLE employees (
id INT NOT NULL,
fname VARCHAR(30),
lname VARCHAR(30),
hired DATE NOT NULL DEFAULT '1970-01-01',
separated DATE NOT NULL DEFAULT '9999-12-31',
job_code INT,
store_id INT
);
```

Suppose that there are 20 video stores distributed among 4 franchises as shown in the following table.

Region Store ID Numbers

North 3, 5, 6, 9, 17
East 1, 2, 10, 11, 19, 20
West 4, 12, 13, 14, 18
Central 7, 8, 15, 16

To partition this table in such a way that rows for stores belonging to the same region are stored in the same partition

```
ALTER TABLE employees PARTITIONBY LIST(store_id) (
PARTITION pNorth VALUES IN (3,5,6,9,17),
PARTITION pEast VALUES IN (1,2,10,11,19,20),
PARTITION pWest VALUES IN (4,12,13,14,18),
PARTITION pCentral VALUES IN (7,8,15,16)
);
```

HASH Partitioning

Partitioning by HASH is used primarily to ensure an even distribution of data among a predetermined number of partitions. With range or list partitioning, you must specify explicitly into which partition a given column value or set of column values is to be stored; with hash partitioning, MySQL takes care of this for you, and you need only specify a column value or expression based on a

column value to be hashed and the number of partitions into which the partitioned table is to be divided. The following statement creates a table that uses hashing on the store_id column and is divided into 4 partitions:

```
CREATE TABLE employees (
id INT NOT NULL,
fname VARCHAR(30),
lname VARCHAR(30),
hired DATE NOT NULL DEFAULT '1970-01-01',
separated DATE NOT NULL DEFAULT '9999-12-31',
job_code INT,
store_id INT
)
PARTITIONBY HASH(store_id)
PARTITIONS 4;
```

If you do not include a PARTITIONS clause, the number of partitions defaults to 1.

CASE STUDY

consider the following **Company data requirements.**

The company is organized into branches. Each branch has a unique number, a name, and a particular employee who manages it.

The company makes its money by selling to clients. Each client has a name and a unique number to identify it.

The foundation of the company is its employee. Each employee has a name, birthday, sex, salary and a unique number.

An employee can work for one branch at a time, and each branch will be managed by one of the employees that work there. We will also want to keep track of when the current manager started as manager.

An employee can acts as a supervisor for other employees at the branch, an employee may also acts as the supervisor for employees at other branches. An employee can have at most one supervisor.

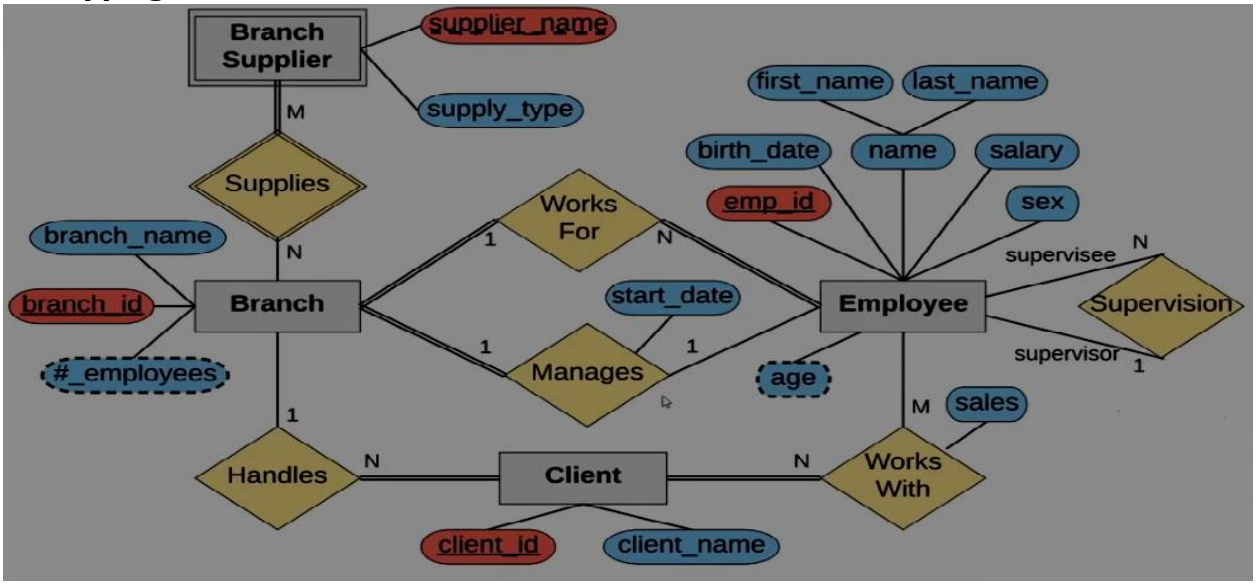
A branch may handle a number of clients, with each client having a name and a unique number to identify it. A single client may only be handled by one branch at a time.
Employees can work with clients, controlled by their branch to sell their stuff. If necessary multiple employees can work with the same client. We will want to keep track of how many francs worth of stuff each employee sells to each client they work with.

Many branches will need to work with suppliers to buy inventory. For each supplier, we will keep track of their name and the type of product they are selling the branch. A single supplier may supply products to multiple branches.

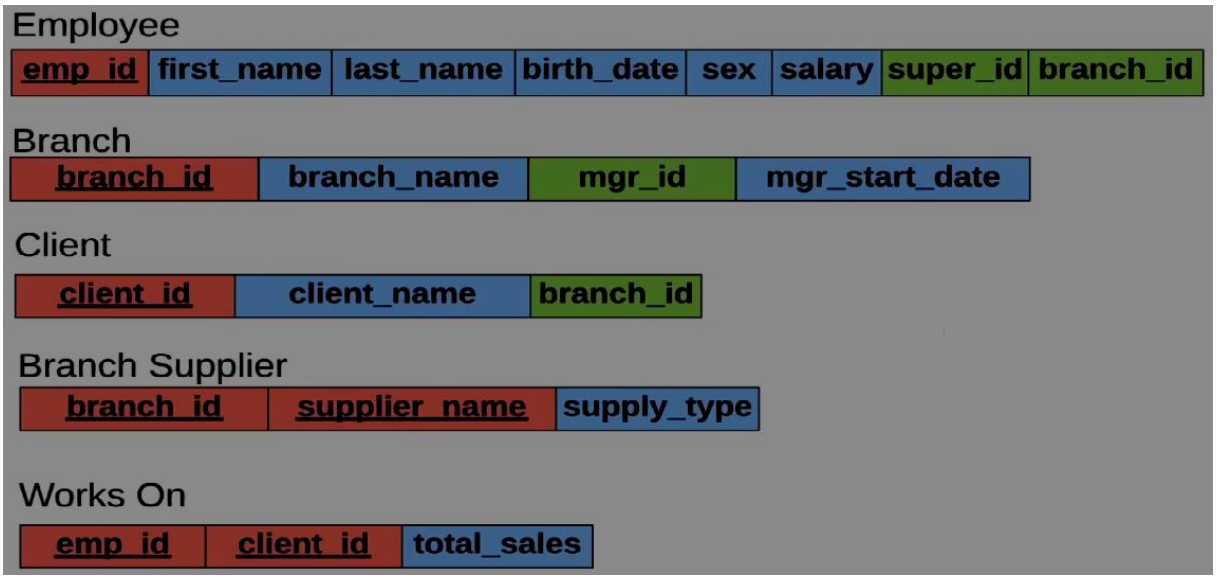
The above information is a **Company data requirements document.** How do you go about converting it into a database schema if you are given a document like this one? The first thing would be to create an Entity-Relationship diagram.

1. Draw an ER diagram that captures this information, by mapping various entities, various type of attributes concerned, relationships and identifying relationships, cardinality ratios, total and partial participations.
2. Covert the diagram above to obtain the relational model
3. implement the physical model to obtain the database.

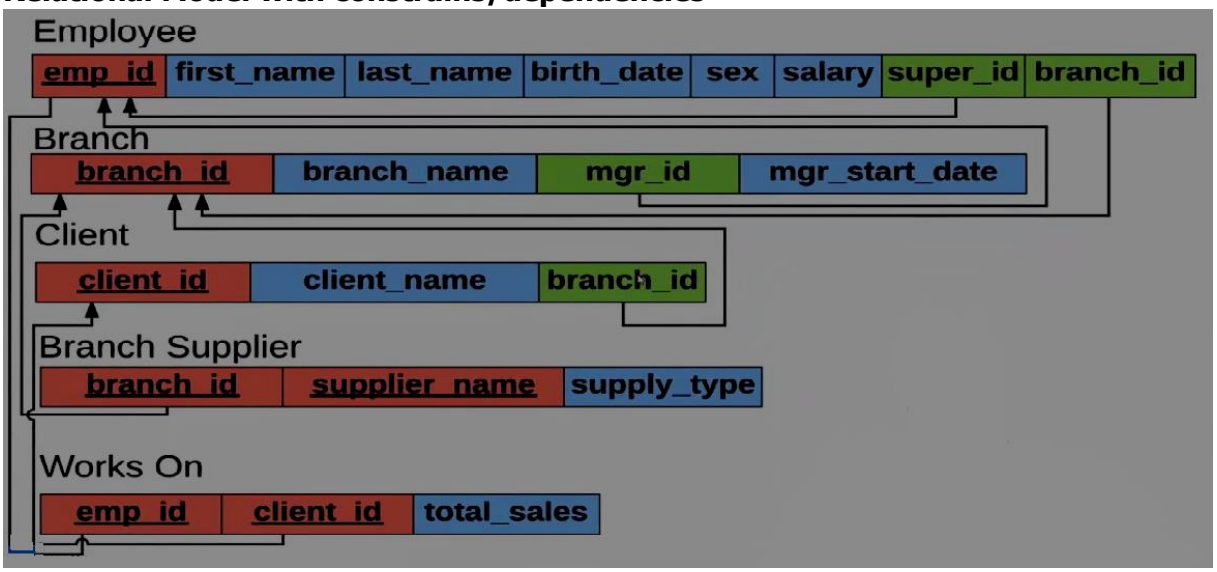
1. Mapping the ER Model below to a Relational Model



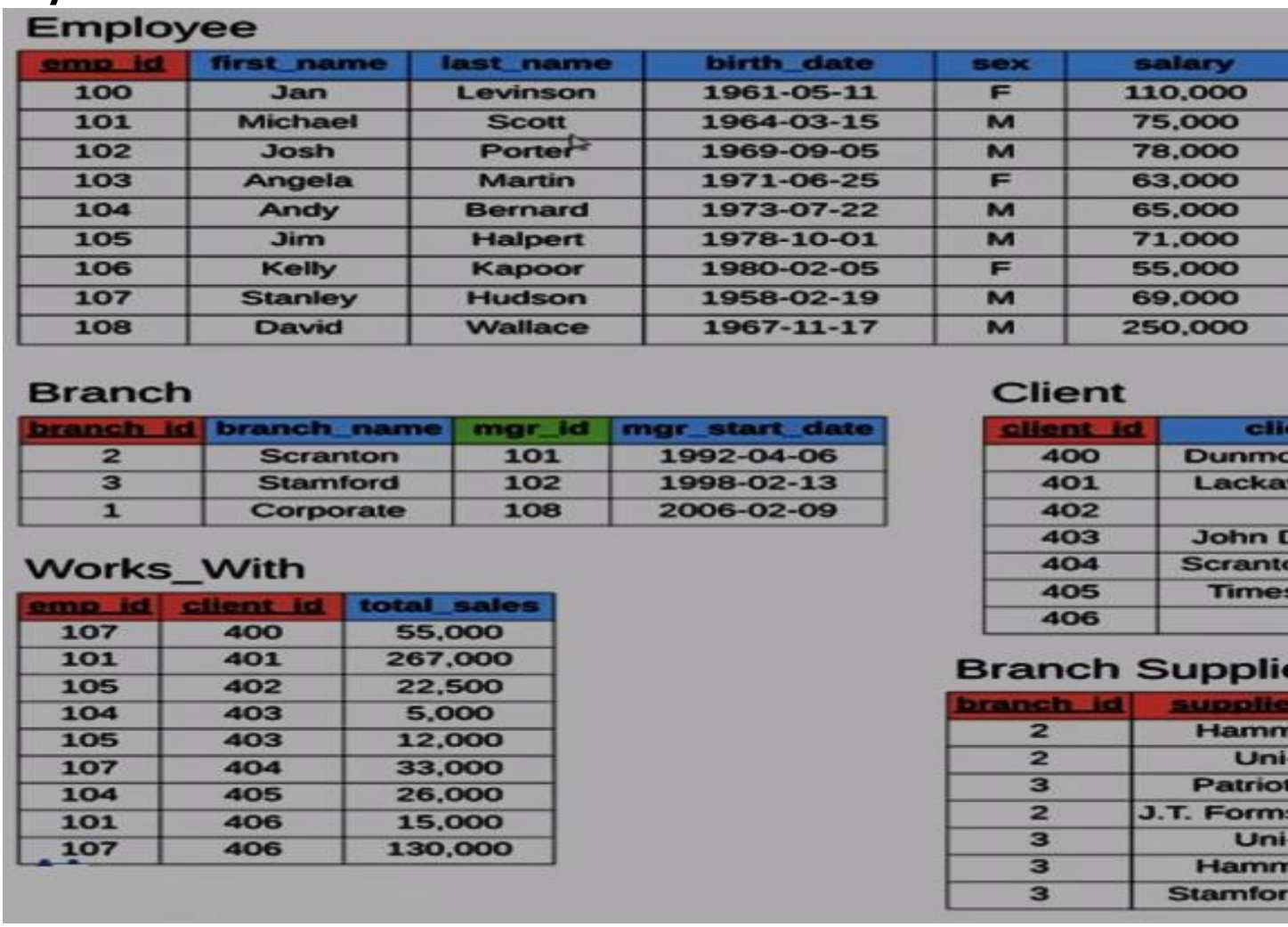
2. Relational Model



Relational Model with constrains/dependencies



Physical database



Employee

emp_id	first_name	last_name	birth_date	sex	salary
100	Jan	Levinson	1961-05-11	F	110,000
101	Michael	Scott	1964-03-15	M	75,000
102	Josh	Porter	1969-09-05	M	78,000
103	Angela	Martin	1971-06-25	F	63,000
104	Andy	Bernard	1973-07-22	M	65,000
105	Jim	Halpert	1978-10-01	M	71,000
106	Kelly	Kapoor	1980-02-05	F	55,000
107	Stanley	Hudson	1958-02-19	M	69,000
108	David	Wallace	1967-11-17	M	250,000

Branch

branch_id	branch_name	mgr_id	mgr_start_date
2	Scranton	101	1992-04-06
3	Stamford	102	1998-02-13
1	Corporate	108	2006-02-09

Works_With

emp_id	client_id	total_sales
107	400	55,000
101	401	267,000
105	402	22,500
104	403	5,000
105	403	12,000
107	404	33,000
104	405	26,000
101	406	15,000
107	406	130,000

Client

client_id	cli
400	Dunmo
401	Lacka
402	
403	John D
404	Scrant
405	Times
406	

Branch Supplie

branch_id	supplie
2	Hamm
2	Uni
3	Patriot
2	J.T. Form
3	Uni
3	Hamm
3	Stamfor

3. Physical implementation of the database

```
CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    first_name VARCHAR (50),
    last_name VARCHAR (50),
    birth_date DATE,
    sex VARCHAR (1),
    salary INT,
    super_id INT,
    branch_id INT
);
```

```
CREATE TABLE branch (
    branch_id INT PRIMARY KEY,
    branch_name VARCHAR (50),
    mgr_id INT,
    mgr_start_date DATE,
    FOREIGN KEY(mgr_id) REFERENCES employee(emp_id) ON DELETE SET NULL
);

ALTER TABLE employee
ADD FOREIGN KEY(branch_id)
REFERENCES branch(branch_id)
ON DELETE SET NULL;

ALTER TABLE employee
ADD FOREIGN KEY(super_id)
REFERENCES employee(emp_id)
ON DELETE SET NULL;

CREATE TABLE client (
    client_id INT PRIMARY KEY,
    client_name VARCHAR (50),
    branch_id INT,
    FOREIGN KEY(branch_id) REFERENCES branch(branch_id) ON DELETE CASCADE
);

CREATE TABLE works_with (
    emp_id INT,
    client_id INT,
    total_sales INT,
    PRIMARY KEY(emp_id, client_id),
    FOREIGN KEY(emp_id) REFERENCES employee(emp_id) ON DELETE CASCADE,
    FOREIGN KEY(client_id) REFERENCES client(client_id) ON DELETE CASCADE
);
```

```
CREATE TABLE branch_supplier (
    branch_id INT,
    supplier_name VARCHAR(50),
    supplier_type VARCHAR(50),
    PRIMARY KEY(branch_id, supplier_name),
   FOREIGN KEY(branch_id) REFERENCES branch(branch_id) ON DELETE CASCADE
);
```

```
INSERT INTO employee VALUES(100, 'DAVID', 'WALANCE', '1967-11-17', 'M', 250000, NULL, NULL);
INSERT INTO branch VALUES(1, 'Corporate', 100, '2006-02-09');
UPDATE employee
SET branch_id =1
WHERE emp_id = 100;
INSERT INTO employee VALUES(101, 'jan', 'Levinson', '1961-05-11', 'F', 110000, 100, 1);

INSERT INTO employee VALUES(102, 'Michael', 'Scott', '1964-03-15', 'M', 75000, 100, NULL);
INSERT INTO branch VALUES(2, 'Scraton', 102, '1992-04-06');
UPDATE employee
SET branch_id =2
WHERE emp_id = 102;

INSERT INTO employee VALUES(103, 'Angela', 'Martin', '1971-06-25', 'F', 63000, 101, 2);
INSERT INTO employee VALUES(104, 'Kelly', 'kapoor', '1980-02-05', 'F', 55000, 101, 2);
INSERT INTO employee VALUES(105, 'Stanley', 'Hudson', '1958-02-19', 'M', 69000, 101, 2);

INSERT INTO employee VALUES(106, 'Josh', 'Porter', '1969-09-05', 'M', 78000, 100, NULL);
INSERT INTO branch VALUES(3, 'Stamford', 106, '1998-02-13');
UPDATE employee
SET branch_id =3
WHERE emp_id = 106;
INSERT INTO employee VALUES(107, 'Andy', 'Bernard', '1973-07-22', 'F', 65000, 102, 3);
INSERT INTO employee VALUES(108, 'Jim', 'Halpert', '1978-01-01', 'F', 71000, 102, 3);

INSERT INTO branch_supplier VALUES(2, 'Hammer Mill', 'Pager');
INSERT INTO branch_supplier VALUES(2, 'Uni-ball', 'Writing Utensils');
INSERT INTO branch_supplier VALUES(3, 'Patriot paper', 'Pager');
INSERT INTO branch_supplier VALUES(2, 'J. T. Forms & &Labels', 'Custum Forms');
INSERT INTO branch_supplier VALUES(3, 'Uni-bal', 'Writing Utensils');
INSERT INTO branch_supplier VALUES(3, 'Hammer Mill', 'Pager');
INSERT INTO branch_supplier VALUES(3, 'Stamford Labels', 'Custom Forms');
```

```
INSERT INTO client VALUES(400, 'Dunmore Highschool', 2);
INSERT INTO client VALUES(401, 'Lackawana Country', 2);
INSERT INTO client VALUES(402, 'FedEx', 3);
INSERT INTO client VALUES(403, 'John Daly Law, LLC', 3);
INSERT INTO client VALUES(404, 'Scranton Whitepages', 2);
INSERT INTO client VALUES(405, 'Times Newspaper', 3);
INSERT INTO client VALUES(406, 'FedEx', 2);


INSERT INTO works_with VALUES(105, 400, 55000);
INSERT INTO works_with VALUES(102, 401, 267000);
INSERT INTO works_with VALUES(108, 402, 22500);
INSERT INTO works_with t VALUES(107, 403, 5000);
INSERT INTO works_with VALUES(108, 403, 12000);
INSERT INTO works_with VALUES(105, 404, 12000);
INSERT INTO works_with VALUES(105, 404, 33000);
INSERT INTO works_with VALUES(107, 405, 26000);
INSERT INTO works_with VALUES(102, 406, 15000);
INSERT INTO works_with VALUES(105, 406, 130000);
```

Querying the database

```
-- Find all employee
SELECT *
FROM employee;

-- Find all employee ordered by salary
SELECT *
FROM employee
ORDER BY salary;

-- Find all employee ordered by salary
SELECT *
FROM employee
ORDER BY salary DESC;
SELECT *
FROM employee
ORDER BY sex, first_name, last_name;


-- Find the first 5 employees in the table
SELECT *
FROM employee
LIMIT 5;
```

-- Find the forename and surenames names of all employees in the table

```
SELECT first_name AS forename, last_name AS surname
FROM employee;
```

-- Find out all the different genders

```
SELECT DISTINCT sex
FROM employee;
```

-- Find the number of employees

```
SELECT COUNT(emp_id)
FROM employee;
```

```
SELECT COUNT(super_id)
FROM employee;
```

-- Find the number of femal employees born after 1st of Jan 1970

```
SELECT COUNT(emp_id)
FROM employee
WHERE sex= ‘F’ AND birth_date >  ‘1970-01-01’;
```

-- Find Average salary

```
SELECT AVG(salary)
FROM employee;
```

-- Find Average salary for male

```
SELECT AVG(salary)
FROM employee
WHERE sex = ‘M’;
```

-- Find the sum of all salary

```
SELECT SUM(salary)
FROM employee;
```

-- Find out how many males and females are there

```
SELECT COUNT(sex) , sex
FROM employee
GROUP BY sex;
```

-- Find the total sales of each salesman

```
SELECT SUM(total_sales) , emp_id
FROM works_with
GROUP BY emp_id;
```

```
-- Find the total sales of each salesman
SELECT SUM(total_sales) , client_id
FROM works_with
GROUP BY client_id;

-- Find any client's who are an LLC
SELECT *
FROM client
WHERE client_name LIKE '%LLC';

-- Find any client's who are an LLC
SELECT *
FROM branch_supplier
WHERE supplier_name LIKE '%label%';

-- Find any employee born in october
SELECT *
FROM employee
WHERE birth_date LIKE '____-02%';

-- Find all employee and their branch
SELECT first_name
FROM employee
UNION
SELECT branch_name
FROM branch;


SELECT client_name, client.branch_id
FROM client
UNION
SELECT supplier_name, branch_supplier.branch_id
FROM branch_supplier;
-- Find all branches and the names of their managers
SELECT employee.emp_id, employee.first_name, branch.branch_name
FROM employee
JOIN branch
ON employee.emp_id = branch.mgr_id;
```

-- Find all branches and the names of their managers

```
SELECT employee.emp_id, employee.first_name, branch.branch_name
FROM employee
LEFT JOIN branch
ON employee.emp_id = branch.mgr_id;


SELECT employee.emp_id, employee.first_name, branch.branch_name
FROM employee
RIGHT JOIN branch
ON employee.emp_id = branch.mgr_id;
```

-- Find the names of all employees who have sold over 30, 000

```
SELECT employee.first_name, employee.last_name
FROM employee
WHERE employee.emp_id IN (
        SELECT works_with.emp_id
        FROM works_with
        WHERE works_with.total_sales > 30000
);
```

-- Find all clients who are handled by the branch that Michael Scott manages, assuming you know his ID

```
SELECT client.client_name
FROM client
WHERE client.branch_id  = (
        SELECT branch.branch_id
        FROM branch
        WHERE branch.mgr_id = 102
LIMIT 1
);

INSERT INTO branch VALUES(4, 'Buffalo', NULL, NULL);

DELETE FROM employee
WHERE emp_id = 102;

SELECT * FROM branch;

DELETE FROM branch
WHERE branch_id = 2;

SELECT * FROM branch_supllier;
```